

个人课本（示例）· 信息学

Git、版本管理与人工智能

学生示例

匿名示例 · 本册用学生的母语

中文版 — Git、版本管理与人工智能

1. 三个工具及其关系

1. Git 和 GitHub：有序保存工作的“仓库”，每个版本都被保存并可以共享。
2. Gemini：可以对话的生成式人工智能（AI）；你给它一段文字（甚至一个 .md 文件），它就为你解释、翻译、总结。
3. NotebookLM：只根据你上传的文档（例如一个 PDF）回答的“辅导老师”，因此它忠于原文。

要记住的核心：Git 负责保存，Gemini 负责讲解，NotebookLM 在原文上帮你复习。它们不是竞争关系，而是配合使用。

课程金规则：人工智能是用来帮助你更好地理解，不是用来抄袭或跳过思考。检验标准：如果你能用自己的话讲出你做了什么、为什么这样做，那才是真正学会了。

2. Git 和 GitHub

1. Git 是一个版本管理系统：它保存工作的历史，因此每一次重要的保存都会留下，可以随时回到过去。
2. GitHub 是让版本化的工作在网络上存放和共享的网站：也就是“网上的 Git”。注意区分：Git 是工具（引擎），GitHub 是在线服务（网上的车库）。
3. 仓库（repository）：项目文件夹，里面有所有文件及其全部历史。
4. 提交（commit）：带标签的保存，为项目在某一时刻“拍照”，并写上一句说明。许多提交排成一列，就构成了项目的历史。
5. 分支（branch）：一条平行的工作线，可以在不破坏主线（main）的情况下尝试修改；把分支合回主线叫做合并（merge）。
6. 拉取请求（Pull Request，PR）：请求把自己的分支合并到主线，通常需要有人先审查。

它属于什么服务：PaaS（平台即服务）——别人为你提供一个现成的在线平台，你直接使用，无需自己搭建和维护服务器。它采用 Freemium 收费模式：基础功能免费，高级功能付费；对学生来说免费部分足够用。

3. 版本管理：版本号 1.2.3

1. major（主版本，第一位）：进行重大的、可能与旧版本不兼容的改动时增加。
2. minor（次版本，第二位）：增加新功能但其余仍可正常使用时增加。

3. patch (修订, 第三位) : 只修复了一个错误、没有新增内容时增加。

例如: `v1.0.0` 第一个可玩版本 → 加入声音 `v1.1.0` → 修复一个错误 `v1.1.1` → 全部重做 `v2.0.0`。

金规则: 已经发布的版本永远不重复使用。哪怕只改一处, 也要先提升版本号再发布。

4. Git 使用场景 (分支图, 用文字描述)

1. 线性开发: 单人、无分支, 提交一个接一个, 历史是一条直线。
2. 一个功能一个分支, 然后合并: 在单独的分支上开发新功能, 主线始终可用; 完成后合并。
3. 两个功能并行: 两个分支同时开始、先后完成; 主线随做随收。
4. 紧急修复 (hotfix) : 长功能进行中出现严重错误, 开一个短分支立刻修复并合并, 再回到原来的功能。
5. 发布版本: 不时在主线上发布版本 (`v1.0`、`v1.1`……直到大改动 `v2.0`) , 每个发布都被“冻结”。
6. 被放弃的分支: 尝试一个想法但从不合并, 主线毫发无损。
7. 从分支再分出子分支: 大功能拆出子分支; 先把子分支并回分支, 再把分支并回主线。
8. 大型整合 (小组项目) : 每人一个分支 (图形、动作、音频、菜单) , 最后都通过 PR 汇入主线, 形成完整版本。
9. 长期维护分支: 主线走向 `v2.0` , 同时为仍在用 `v1` 的人保留一个只做修复的并行分支 (`v1.1`、`v1.2`) 。
10. revert (撤销错误改动) : 不删除历史, 而是加一个新提交来撤销错误的那个, 如实留下痕迹。
11. 合并冲突并解决: 两个分支改了同一行, Git 无法自动决定, 由人来选择保留哪一个并完成合并。
12. 五分支两发布的团队: 主线发布版本, 四个分支并行、先后汇入。
13. 五层嵌套分支: 把复杂工作一层层拆分, 每层并回上一层, 像套娃一样。
14. 同时维护两个主版本: `v1.x` 和 `v2.x` 各有一条维护分支同时存在。
15. 真实历史: 发布、长功能与子分支、紧急修复、撤销、带冲突的合并, 全部同时发生。

5. Markdown 文件 (.md)

1. Markdown (扩展名 `.md`) 是纯文本, 只用少量符号排版: `#` 表示标题, `**` 表示加粗, 以 `1.` 开头表示编号列表。
2. 它到处都能阅读, 没有沉重的图片, 只有内容。因此人工智能能很好地读取它: 课程把讲义也提供为 `.md`, 正是为了交给 Gemini。

6. Gemini : 生成式 AI , 以及如何用 .md 文件

1. Gemini 是生成式 AI : 你写一个请求 (提示, prompt) , 它生成回答——解释、翻译、总结、出复习题。
2. 注意 : AI 也会出错或编造 (“幻觉”) , 它说的内容要核对, 尤其是日期、数字和来源。
3. 使用步骤 : 在浏览器打开 `gemini.google.com` (用学校账号) ; 打开 .md 文件, 全选并复制 ; 粘贴到 Gemini , 写下你的请求。

选哪个版本 : Flash (快速, 日常用)、Pro (更强, 复杂长文本)、推理版 (先“思考”再回答, 适合难题)。实用规则 : 先用 Flash, 若回答复杂或有误, 再换 Pro 或推理版。看它“用途” (快 / 强 / 会推理) , 而不是只看数字。

7. NotebookLM : 基于来源的辅导, 以及如何用 PDF

1. NotebookLM 只根据你上传的文档回答 : 上传一个 PDF, 它就用那个 PDF 回答, 不在外面编造。
2. 因此它很适合在一份确定的资料上学习和复习, 忠于原文。
3. 步骤 : 打开 `notebooklm.google.com` (学校账号) ; 点“新建”, 再点“上传”, 选一个 PDF ; 在聊天框里提问, 例如“给我这份文档的简单总结”。

8. 完整流程 : 三个环境一起用

1. 课程材料放在 Git/GitHub (已版本化) 。
2. 从那里下载 PDF (阅读/打印) 或 .md 文件 (给 AI) 。
3. 把 .md 交给 Gemini , 让它讲解或翻译。
4. 把 PDF 上传到 NotebookLM , 在原文上提问、复习。
5. 完成的作业在 Classroom 上交。

9. 负责任地使用 AI

1. 理解, 而非抄袭 : AI 是学习的帮手, 不是跳过工作的捷径。检验 : 能用自己的话讲出来。
2. 始终核对 : AI 可能出错或编造, 要核实它说的内容。
3. 隐私与数据 : 不要把个人、同学或家庭的数据输入这些助手。

评价示例 : 分数 90/100 — 作业完整、正确, 对 Git 与版本管理理解清晰, 能用自己的话解释。
(匿名示例)